AutoGrader

A Rule-Driven Evaluator for C Programs

Final Project Report
Design and Analysis of Algorithms Laboratory
IIT Patna — M.Tech Artificial Intelligence

Team member	Primary responsibility
Mudavath Vikas Naik	Compilation/execution runner, timeout and error handling, runner tests, integration
Sudeep Raj	Correctness evaluation and sanitizer-based memory-safety checks
Rohit Raj	Static-analysis and source-quality analyzers
Dhruv Kumar	External rubric/rules, score aggregation, coordination and reporting

Executive summary

AutoGrader is a modular, rule-driven C-program grading application. It accepts a C source file or supported project ZIP, builds it using an allow-listed toolchain, performs bounded execution, invokes separate metric analyzers, aggregates scoreable metrics using an external JSON rubric, and produces machine-readable and student-readable reports. The Streamlit interface supports instructor configuration, test-suite import, individual and batch grading, evidence inspection and report export.

The implementation is assignment-agnostic at the build/execution layer, not at the correctness-oracle layer. The instructor must provide trusted expected outputs or a verified reference oracle for each assignment. It intentionally does not assume that every submission is an addition or factorial program.

1. Assignment requirements and implementation

Assignment requirement	Implementation response
Safe execution	GCC compilation, wall-clock timeout, bounded output, process-group cleanup, POSIX address-space/CPU limits where suppo
External editable rules	rules/rules.json stores time/memory limits, enabled analyzers, thresholds, weights, parameters and grading policy.
Separate analyzers and weighted grade	Dedicated analyzers for correctness, syntax, structure, modularity, logical edge cases, complexity, memory safety, readability
Unattended reproducible workflow	Coordinator runs first-pass checks, dispatches enabled analyzers, aggregates results and writes JSON/text reports with sourc
Specific failure feedback	Correctness evidence records test input, expected output, actual output, stderr, status and duration for each supplied trusted o
Copying / LLM suspicion	Normalized token-pair similarity compares against supplied reference submissions/corpus and emits a human-review flag. It is
Usefulness and limitations	Four concrete usefulness examples and four concrete limitations with unreliable-verdict scenarios are documented in docs/US

2. Architecture and workflow

Instructor configuration → submission upload → build/compile → first-pass smoke check → enabled specialist analyzers → weighted score calculation → JSON/text report and UI evidence.

The first-pass check prevents execution-based analyzers from running after a failed smoke execution. A timeout, crash, runtime error or output-limit failure marks execution as blocked, skips correctness verification and applies the configured diagnostic score cap. Source-only analyzers may still provide diagnostic information.

3. Analyzer overview

Metric	Method and interpretation
Correctness	Runs instructor-authored tests and compares stdout / expected generated files. Reports failing input and expected/actual output. Requ
Syntax	Compiler diagnostics using configurable GCC flags; warning policy is rubric-controlled.
Structure	Static source heuristics for structural indicators; not a full formal C parser.
Modularity	Function-count and helper-function heuristics.
Logical	Optional assignment-specific edge cases; disabled by default to avoid applying unrelated generic cases.
Complexity	Loop/decision-token structural estimate. Heuristic only; not a formal proof or robust empirical timing curve.
Memory safety	ASan/UBSan instrumented runs when supported; only detects defects triggered by executed paths.
Readability	Deterministic offline source heuristic; not an LLM judgement.
Originality	Normalized token-pair similarity against supplied corpus/references; review lead only.

4. Assignment independence and test oracle

Supported modes include a standalone .c file, recursively discovered multi-file C project ZIP, conventional single .l plus single .y/.yy Flex/Bison project, and restricted custom argv build steps. Uploaded shell scripts and Makefiles are not run. Project ZIP extraction rejects path traversal and enforces entry/size limits. Test cases support stdin, command-line arguments, staged input files, expected stdout and expected generated-file contents.

The grader cannot infer intended semantics or output format from arbitrary C source alone. A compiler assignment needs trusted input programs and expected compiler output (or a verified reference compiler). If a program writes output files, expected_files must be configured. No generic test generator can reliably invent expected answers for an unknown problem.

5. Validation and sample outputs

Final packaging validation: Python compileall passed and 28 automated unit/integration tests passed. Tests cover output comparison, rubric validation, weighted scoring, disabled/missing metrics, infinite-loop timeout and score cap, address-space limits, sanitizer detection on the available host, source similarity normalization, project ZIP builds, argv/file-based tests and shell/tool allow-listing.

Sample submission	Overall score	Status	Interpretation
Correct sample	92.78 / 100	complete	Trusted cases pass; quality metrics reduce the total below 100.
Wrong-answer sample	74.04 / 100	complete	Some tests fail; inspect per-case evidence.
Compile failure	0 / 100	compile_failure	Compilation did not succeed.
Infinite loop	20 / 100	execution_blocked	Smoke test timed out; correctness not verified; cap applied.
Memory-error sample	55.56 / 100	complete	Inspect sanitizer evidence; depends on runtime and test path.

These scores are generated from the packaged sample suite and rubric; they are demonstrations, not universal expected grades. Streamlit browser interaction was not launched in the isolated packaging environment because Streamlit was not installed there. Complete the manual UI check in the target WSL environment.

6. Usefulness in real-life settings

1. Large programming courses: grade hundreds of submissions against a common trusted suite and rubric, reducing repetitive compile/run work.
2. Student debugging: show a reproducible failing input, expected output, actual output and runtime diagnostic.

3 Code-quality screening: combine functional results with compiler warnings, structure, modularity and readability indicators.

4 Originality triage: compare submissions with an instructor-supplied corpus and prioritize high-similarity pairs for human review.

7. Limitations and unreliable verdict examples

1 Finite tests cannot prove correctness: a program passes all supplied cases but fails on an untested boundary value or unusual whitespace.

2 Sanitizers have path-coverage limits: a use-after-free exists only in a branch not reached by tests, so ASan reports no issue.

3 Similarity can mislead: two students independently write a short required loop (false positive), or copied code is heavily refactored (false negative).

4 Static heuristics can misestimate properties: macros, recursion, data-dependent loops or unusual control flow can fool complexity/structure heuristics.

8. Security and deployment boundary

The runner applies time, memory, output and process-cleanup controls where supported, but it is not a hardened sandbox. Untrusted submissions should run inside an institution-managed disposable Linux VM/container with non-root privileges, network isolation, restricted filesystem mounts, cgroup CPU/memory/PID limits and appropriate seccomp/AppArmor controls. Do not mount host credentials or Docker sockets into the grading environment.

9. Installation and execution

On Ubuntu/WSL: sudo apt update && sudo apt install -y python3 gcc build-essential cppcheck flex bison. From the project root, install the UI dependency with pip install -r requirements-ui.txt, then launch with streamlit run app.py. For CLI grading: python3 -m src.main --source PATH --tests PATH --rules rules/rules.json --output-dir reports/demo. Run tests with python3 -m unittest discover -s tests -v.

10. Final pre-submission checklist

- Open Streamlit in WSL and confirm no startup traceback.
- Confirm the Test suite tab starts empty and has no unrelated hard-coded task template.
- Import the actual assignment trusted tests/reference outputs.
- Run correct, wrong-answer, compile-error and infinite-loop submissions.
- Inspect failing input / expected / actual output and verify the infinite-loop cap.
- Export JSON and text reports and open them.
- Record the 15–20-minute video with actual application demonstration.
- Submit source repository, installation guide, report, rules and sample reports.

Prepared from the supplied assignment brief and packaged implementation. Team members should review the report, understand the code they present, and be prepared to explain design choices and limitations.